



Raspodijeljene i nerelacijske baze podataka

VJEŽBA 04: pohranjene procedure

Programske strukture

```
BEGIN { sql_statement |  
statement_block }  
END
```

Grupiranje skupine izraza. Ključne riječi za „*control-of-flow*”

```
IF (uvjet)  
    PRINT 'radnje...'  
ELSE IF (uvjet2)  
    PRINT 'radnje2...'  
ELSE  
    PRINT 'radnje3...';
```

IF-ELSE IF-ELSE struktura. Ako ima više od jedne radnje potrebno ih je obuhvatiti s BEGIN-END blokom.

```
WHILE (uvjet)  
BEGIN  
    {...izrazi...}  
END
```

WHILE petlja. **Sve** izraze unutar petlje potrebno je obuhvatiti s BEGIN-END blokom.
BREAK izraz zaustavlja petlju
CONTINUE nastavlja provođenje

Primjeri

```
DECLARE @i INT
SET @i=1
WHILE ( @i <= 10)
BEGIN
    PRINT '@i = ' + CONVERT(VARCHAR,@i)
    --SELECT '@i = ' + CONVERT(VARCHAR,@i)
    SET @i = @i + 1
END
```

Komentirajte prethodnu liniju. A ovu izvršite. Koja je razlika?

```
DECLARE @i INT =1
IF (@i%2=0)
    PRINT 'Broj ' + CONVERT(VARCHAR,@i) + ' je paran'
ELSE
    PRINT 'Broj ' + CONVERT(VARCHAR,@i) + ' je neparan'
```

Bi li radilo da je izjava unutar grananja stavljena u BEGIN-END blok?

Primjeri

```
DECLARE @datum_rodjenja date;
DECLARE @dob int;
SET @datum_rodjenja = '1.1.1990'
SET @dob =year(getdate())-year(@datum_rodjenja);

IF @dob < 18
    PRINT 'maloljetnik';
ELSE IF @dob < 50
    PRINT 'Odrasla osoba';
ELSE
    PRINT 'Senior';
```

Zadatak

- U prošloj vježbi je trebalo napraviti tablicu
 - Tablicu: `production.product_log`
 - Trigger: `production.trg_product_log`
- ZADATAK:
 - Unesite retke u tablicu `products` (koristeći while petlju)
 - Trek1, brand_id=9, category_id=6, 2023, 999.99)
 - Trek2, brand_id=9, category_id=6, 2023, 1000.99)
 - Izbrišite redak za proizvod naziva Trek2
- Prije i nakon svake akcije provjerite tablicu `product_log` i objasnite što se dogodilo!

```
WHILE (uvjet)
BEGIN
    {...izrazi...}
END
```

Zadatak

- Pripremit upit za brisanje dva unesena retka u tablicu `production.products`
- Prije izvršavanje eksplicitno pokrenite `BEGIN TRANSACTION;`
- Provjerite stanje u tablicama:
 - `production.products`
 - `production.product_log`
- Ako napravimo `ROLLBACK TRANSACTION;` hoće li se stanje u obje tablice vratiti na ono prije transakcije?

Rješenje

```
BEGIN TRANSACTION;  
DECLARE @i INT =1  
WHILE (@i<=2)  
BEGIN  
    INSERT INTO production.products VALUES  
    (  
        'Trek'+cast(@i as varchar),  
        9,  
        6,  
        2023,  
        999.99+@i  
    );  
    SET @i+=1  
END
```

Usporedite izraze!

```
insert into production.products values  
(  
    'Trek',  
    9,  
    6,  
    2023,  
    999.99  
)  
GO 2
```

Zadatak

- Onemogućiti (disable) okidač iz prošle vježbe (`sales.trg_stock_upd`)
- Napraviti novi okidač prije unosa podataka (INSTEAD OF INSERT) `sales.trg_order_item_ins` gdje treba
 1. Napraviti provjeru ima li dovoljno na skladištu artikla koji se naručuje
 2. a) Ako ima dovoljno izvršiti insert i update
b) Ako nema, poslati odgovarajuću poruku

```
--primjer poruke o grešci  
RAISERROR ('Nema dovoljno proizvoda na zalihi' ,10,1)
```



```
--provjera ima li dovoljno na skladištu proizvoda (količina na skladištu - naručena količina)
IF (select quantity from production.v_products_stock where product_id=@prod_id and
store_id=@store_id)-(SELECT i.quantity FROM inserted i)<0
BEGIN
    RAISERROR ('Nedovoljno proizvoda na skladištu' ,10,1)
END
ELSE
BEGIN
    --izvršiti unos i ažuriranje
END
```

Zadaci za samostalni rad

- **Pripremite** (ne izvršiti) sljedeće upite:
 1. Podaci o stanju na zalihi proizvoda (product_id=6), trgovine (store_id=1) iz pogleda v_products_stock
 2. Podaci o proizvodima na narudžbi 1 (order_id=1)
 3. Ažurirajte stanje na zalihi proizvoda (product_id=6), trgovine (store_id=1) na 10 dostupnih komada
 4. Unos nove stavke narudžbe uz eksplicitni početak transakcije (order_id=1) za product_id=6, store_id=1, 5 komada

```
--1. Podaci o stanju na zalihi proizvoda (product_id=6), trgovine (store_id=1) iz pogleda v_products_stock
select * from
production.v_products_stock
where product_id=6;

--2. Podaci o proizvodima na narudžbi 1 (order_id=1)
select * from
sales.order_items
where order_id=1;

--3. Ažurirajte stanje na zalihi proizvoda (product_id=6), trgovine (store_id=1) na 10 dostupnih komada
update production.stocks
set quantity=10
where product_id=6
and store_id=1;

--4. Unos nove stavke narudžbe (order_id=1)
begin transaction;
insert into sales.order_items values (
1,
(select max(item_id)+1 from sales.order_items where order_id=1),
6,
5,
(select list_price from production.products where product_id=6),
0
);
```

Zadaci i pitanja

1. Izvršite prva dva upita (stanje na zalihamama i narudžbe 1)
2. Ažurirajte stanje na zalihi (3. upit)
3. Izvršite unos (4.upit) 5 komada
 - a) Koliko redaka se ažuriralo u kojoj tablici?
 - b) Provjerite sa prva dva upita stanje na zalihi i stanje stavki narudžbe 1
 - c) Ako napravimo ROLLBACK hoće li se sve promjene vratiti na stanje prije početka transakcije?
 - d) Napravite `ROLLBACK TRANSACTION;` i opet provjerite stanja u obje tablice (upiti 1 i 2)
 - e) Opet ažurirajte stanje na zalihi (3. upit) ali s 11 komada
 - f) Objasnite što se dogodilo!

Zadaci i pitanja

- Ažurirajte stanje na zalihi na 100 komada
- Izvršite sljedeći upit

```
--4. Unos nove stavke narudžbe (order_id=1)
BEGIN TRANSACTION;
INSERT INTO sales.order_items VALUES (
1,
(SELECT MAX(item_id)+1 FROM sales.order_items WHERE order_id=1),
6,
5,
(SELECT list_price FROM production.products WHERE product_id=6),
0
)
GO 100
```

Čemu služi GO 100

Pohranjene procedure (eng. stored procedures)

```
CREATE PROCEDURE [schema_name].naziv_procedure
    @parameter_name data_type,
    . . . .
    parameter_name data_type
AS
BEGIN
    -- SQL statements
    -- SELECT, INSERT, UPDATE, or DELETE statement
END
```

```
EXEC [schema_name].naziv_procedure;
```

Poziv procedure (bez parametara)

Primjer

- Napravite upit za dohvat svih podataka iz pogleda

`production.v_products_stock`

```
SELECT * FROM v_products_stock
```

- Napravite proceduru `sp_get_product_stock` koja kao rezultat vraća isti upit

```
CREATE PROCEDURE  
production.sp_get_product_stock  
AS  
    BEGIN  
        SELECT * FROM production.v_products_stock  
    END
```

```
EXEC production.sp_get_product_stock;
```

Primjer

- Promijenite proceduru tako da vraća sortirane podatke prema nazivu proizvoda

```
ALTER PROCEDURE sp_get_product_stock  
AS  
    BEGIN  
        SELECT * FROM production.v_products_stock ORDER BY 2  
    END
```

Da se procedura poziva s aplikacijske razine bi li projekt trebalo mijenjati?

Primjer

- Napravite proceduru `production.sp_filter_products_name_stock` tako da prima parametar tipa string prema kojem će se filtrirati proizvodi po nazivu.

```
CREATE PROCEDURE sp_filter_products_name_stock (@proizvod VARCHAR(50))
AS
BEGIN
    SELECT * FROM production.v_products_stock WHERE product_name like '%' + @proizvod + '%'
END
```

```
EXEC production.sp_filter_products_name_stock 'trek';
```

Primjer

- Napravite proceduru `production.sp_ret_cnt_products` koja **vraća** broj proizvoda koji se nalaze u zadanoj trgovini (ulazni parametar id trgovine)

```
CREATE PROCEDURE sp_ret_cnt_products (@store_id INT, @cnt INT OUTPUT)
AS
BEGIN
    SET @cnt=count(product_id) FROM production.v_products_stock WHERE store_id =@store_id
RETURN @cnt
END
```

```
EXEC production.sp_ret_cnt_products (1);
```

Zadaci

- Na stranicama kolegija preuzmite i riješite zadatke v_4 te ih predajte kao zadaću.